

AKI's Poona College, Arts, Commerce & Science, Pune

Mini Project Documentation

Pizza Mania (Online Pizza Ordering website)

Created by

Mohammad Rashid [Roll No. 8933]
&
Santino Mier Tot [Roll No. 8948]

Course Name

S.Y. Bsc (computer science), Batch B1

Introduction

Problem Identification:

The Traditional Pizza ordering process is mostly manual, where customers have to visit the store or place and order through phone calls. This often leads to errors in orders taking, delays in service, and inconvenience for customers. Moreover, customers do not get up the proper way to view the complete menu and explore options easily.

Problem description:

The current system lacks an easy-to-use platform for online pizza ordering. Customers face difficulties such as limited access to the full menu, lack of interactive features, and track orders. These issues highlight the need for a simple, user-friendly website that improves the overall customer experience.

Purpose:

The purpose of this project is to develop a user-friendly website for Pizza Mania where customers can easily view the menu, place orders online and contact the store without any difficulty. It aims to provide convenience, reduce errors in ordering, and enhance the overall customer experience through a simple digital platform.

Objectives:

- To design a simple and attractive website using HTML, CSS and JavaScript.
- To allow customers to view the menu and place orders online with ease.
- To improve customer experience by providing quick access to contact and order details.

Technology:

- HTML : for creating the structure of webpages.
- CSS: for styling layout design.
- JavaScript: for adding interactivity and client side logic.
- [Node.js](#) : for server side development and handling requests.
- [Express.js](#): for building backend APIs and routing.
- MongoDB: for storing Order details.

Modules of the Project:

- **Home Module** : display welcome page with restaurant intro and highlights.
- **About us Module** : provides details about Pizza Mania, service and values.
- **Menu Module** : Show list of Pizza's and order items with descriptions and prices.
- **Order Module**: Allows users to select items, add to cart, and place orders.
- **Contact us Module** : Provides contact details and inquiry form for users.
- **Admin Module** : for managing menu, orders, customers details.

Requirement Specification

Software and Hardware Requirement:

S/W :

- OS: Windows
- Frontend : HTML, CSS, JS
- Backend: [Node.js](#), [Express.js](#)
- Database: MongoDB
- Editor : VS code
- Browser: Chrome

H/W :

- Processor : intel i3 or above.
- RAM: 4 GB minimum (8 GB recommended)
- Hard disk : 250 GB or above..

Functional Requirements

A. User / Customer Side:

- **Registration & Login:** Users must be able to create an account and log in securely.
- **Menu Browsing:** The system must display a list of available pizzas with their descriptions and prices.
- **Cart Management:** Users should be able to add multiple items to a cart and view the total price.
- **Order Placement:** Users must be able to submit their order and provide billing/contact details.
- **Contact/Inquiry:** Users should be able to send messages or feedback through a contact form.

B. Admin Side (Based on your Admin Panel files):

- **Dashboard:** Provide a summary of total orders, revenue, and registered users.
- **Product Management:** Admin must be able to **Add, Edit, and Delete** pizza items from the menu.
- **Order Tracking:** Admin must be able to view all customer orders and update their status (e.g., Pending, Delivered).
- **User Management:** Admin must be able to view a list of all registered customers.
- **Enquiry Management:** Admin must be able to read and manage messages sent through the contact form.

Methodology

1. Database Design (NoSQL Approach)

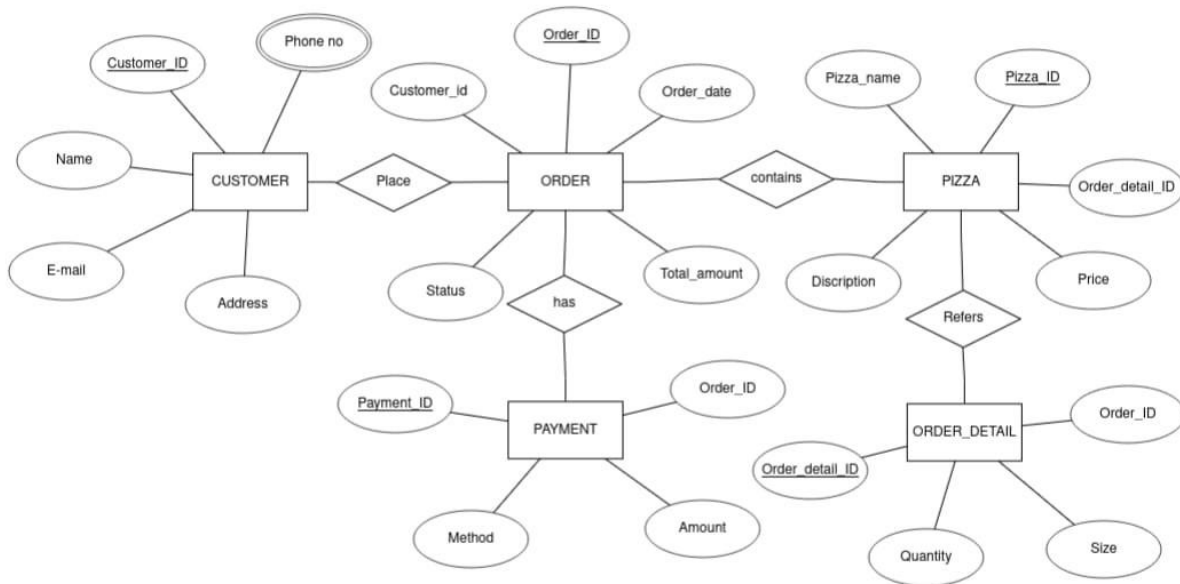
- **Concept:** Uses **MongoDB Collections** instead of traditional tables to store data in flexible JSON-like formats.
- **Key Collections:**
 - **Users:** Stores Name, Email, Password, and Role (Admin/Customer).
 - **Products:** Stores Pizza details (Name, Price, Category, Image).
 - **Orders:** Tracks transactions (User ID, Items, Total, Status).
 - **Enquiries:** Stores messages from the "Contact Us" form.
- **Linking:** Uses **Referencing** (storing User_ID in Orders) to connect data without duplication.

2. System Design (Client-Server Architecture)

- **Frontend (The Client):**
 - **HTML/CSS:** For a clean, "Soft Aesthetic" responsive UI.
 - **JavaScript:** Handles interactive features like "Add to Cart" and Fetch APIs for smooth page updates.
- **Backend (The Server):**
 - **Node.js & Express.js:** Manages the logic, API routing (e.g., /api/orders), and server security.
 - **Middleware:** Validates Admin access and processes data.
- **Admin Module:**
 - A visual **CRUD Dashboard** that allows the owner to Add, Edit, or Delete menu items and manage orders without writing code.
- **Integration:**
 - **API Flow:** The Frontend talks to the Node.js Server, which performs operations on the MongoDB Database and returns the results.

Database Design

ER-Diagram:



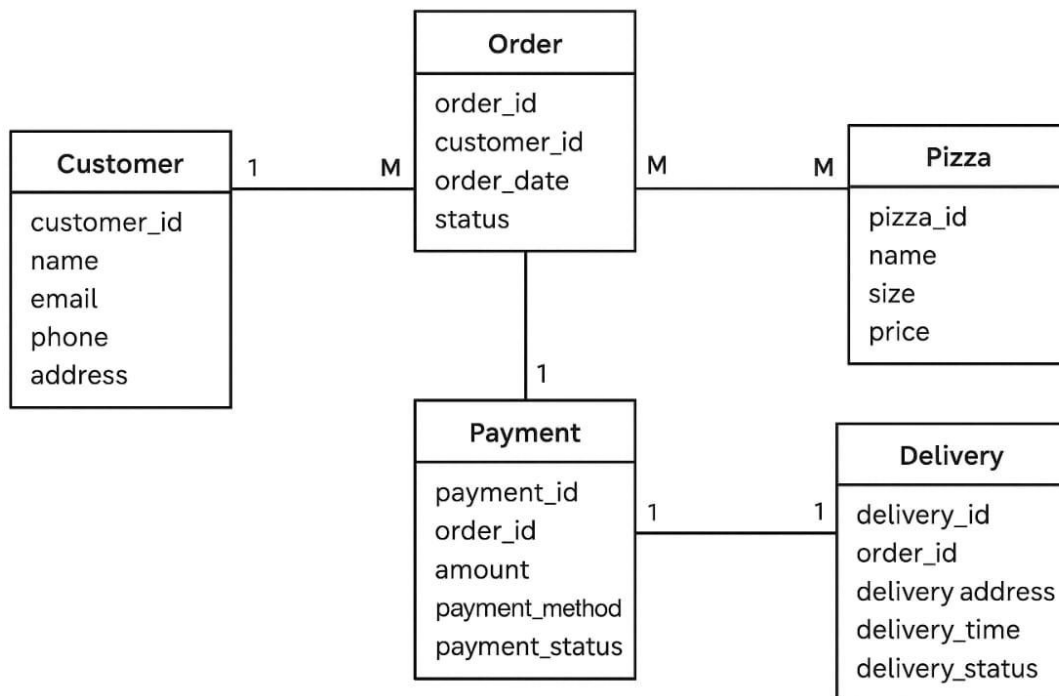
Identifying all tables, fields, relationship between tables etc.

Relationship Name	Entity A	Entity B	Type	Description
Places	Customer	Order	1-to-Many	One customer can place many orders
Contains	Order	Order_Detail	1-to-Many	One order can contain multiple order details
Includes	Pizza	Order_Detail	1-to-Many	One pizza can be included in many order details
Manages	Admin	Pizza	1-to-Many	Admin can manage many pizzas/menu items

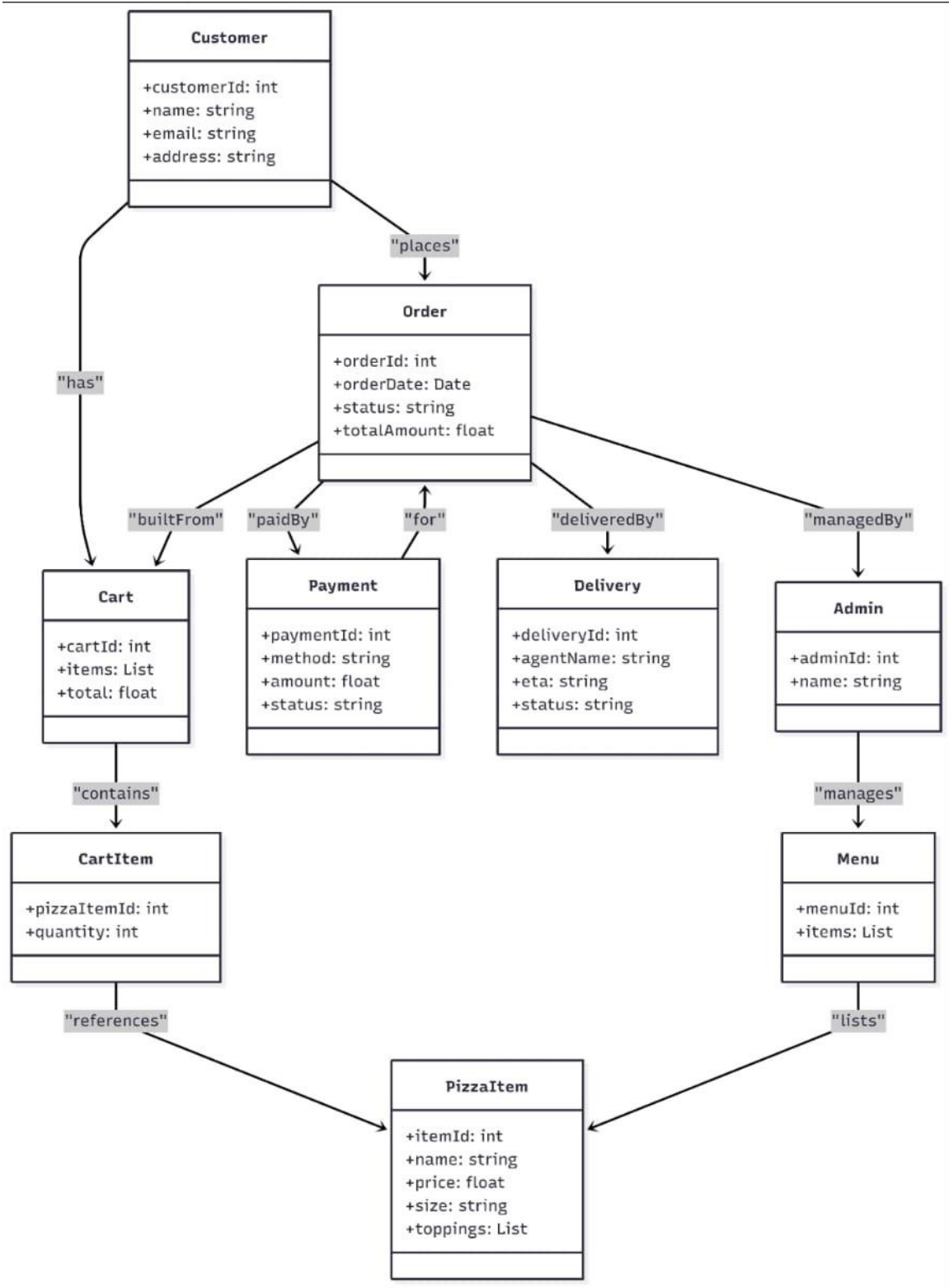
Handles	Admin	Order	1-to-Many	Admin can view/manage multiple orders
Sends	Customer	Contact	1-to-Many	Customer can send multiple contact inquiries

System Design

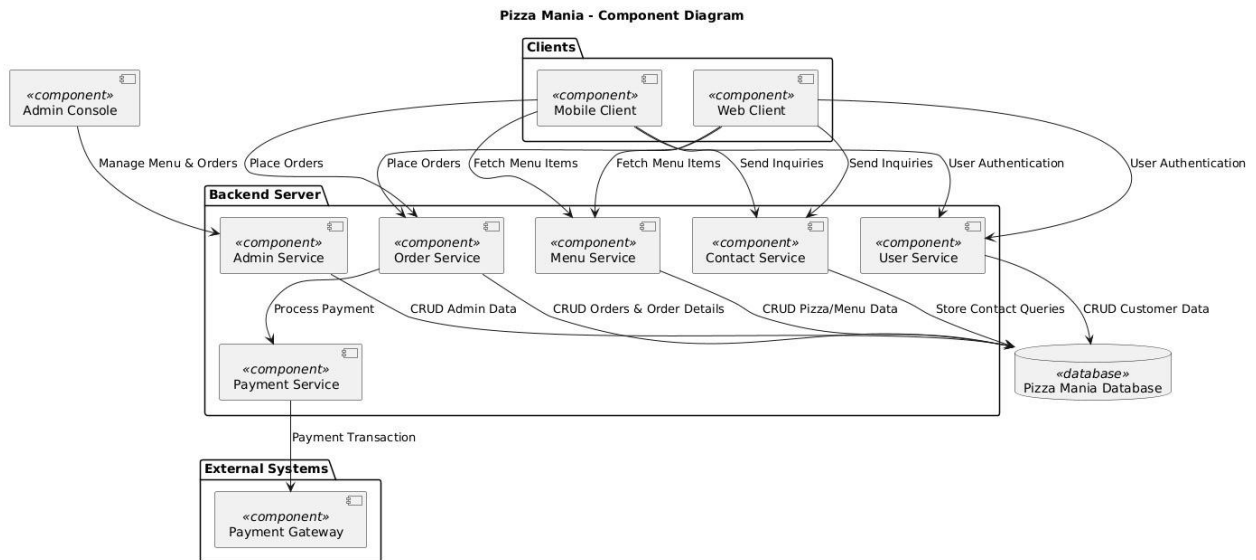
Class Diagram:



Object Diagram:

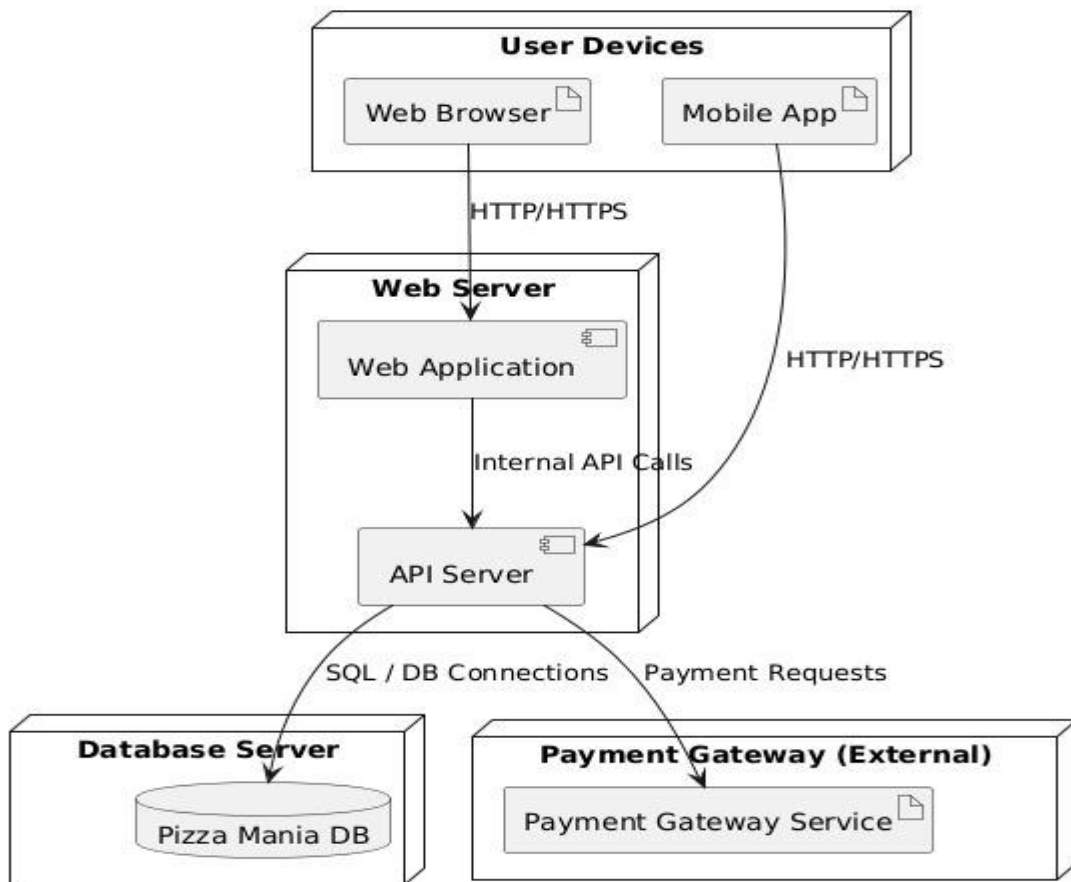


Component Diagram:



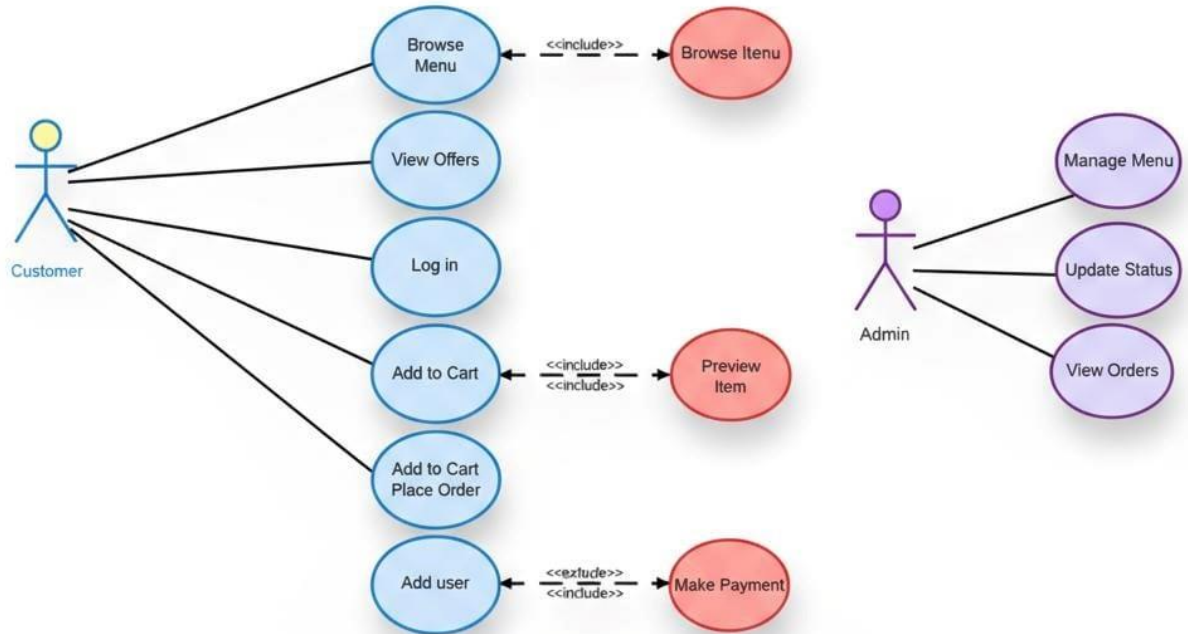
Deployment Diagram:

Pizza Mania - Deployment Diagram

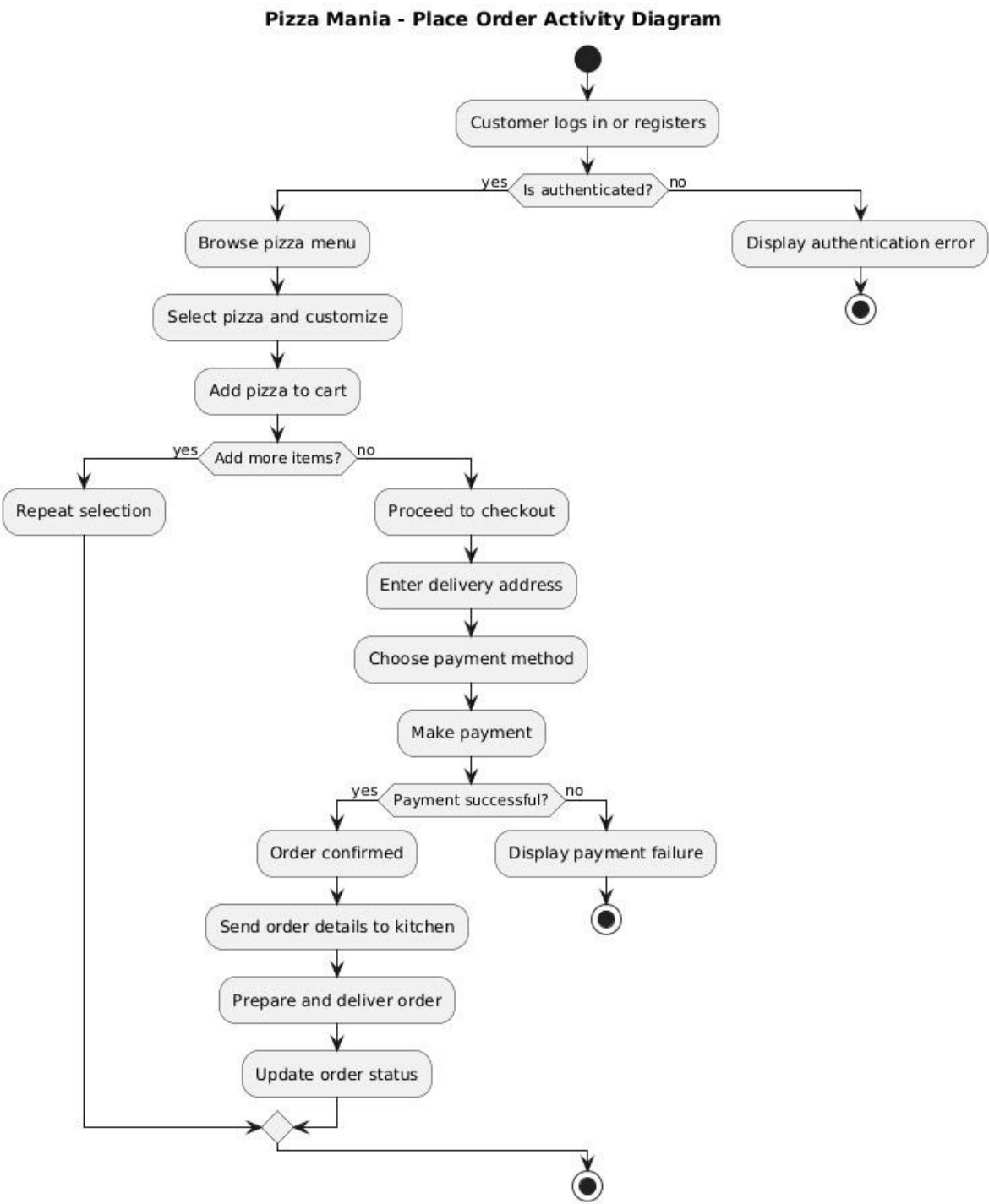


Use Case Diagram:

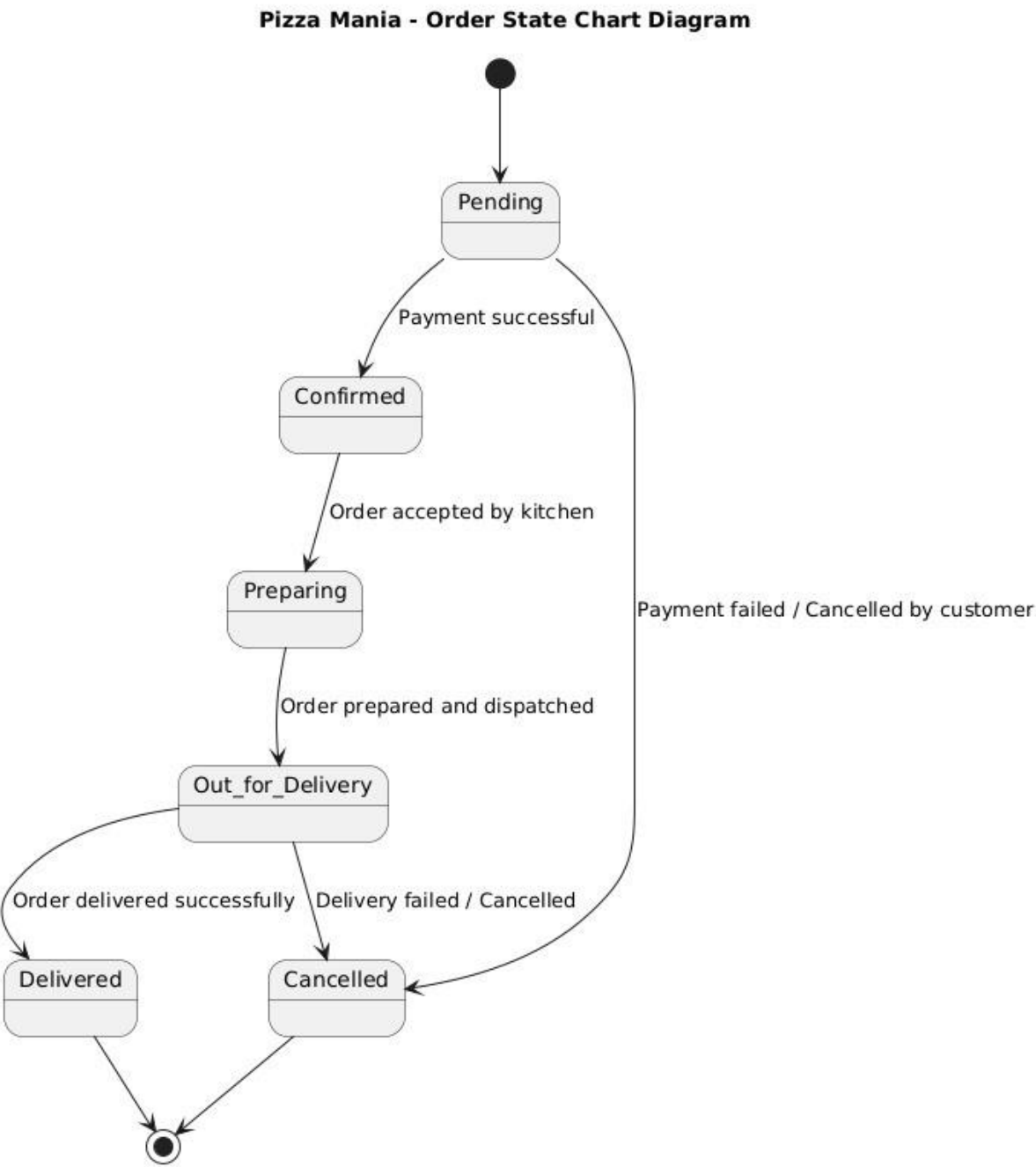
Website Use
Case Diagram



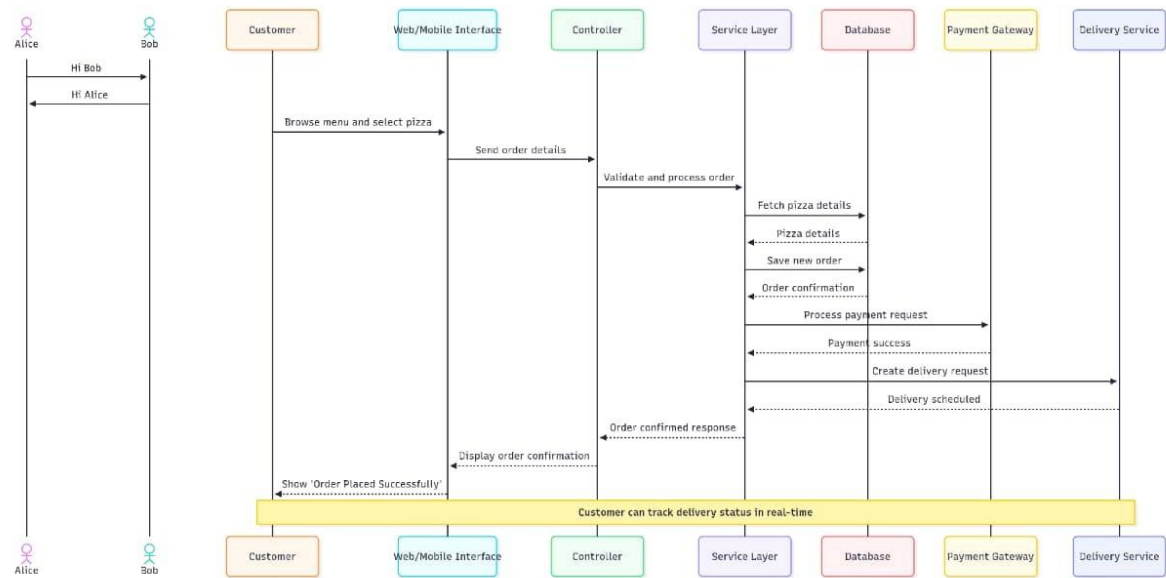
Activity Diagram:



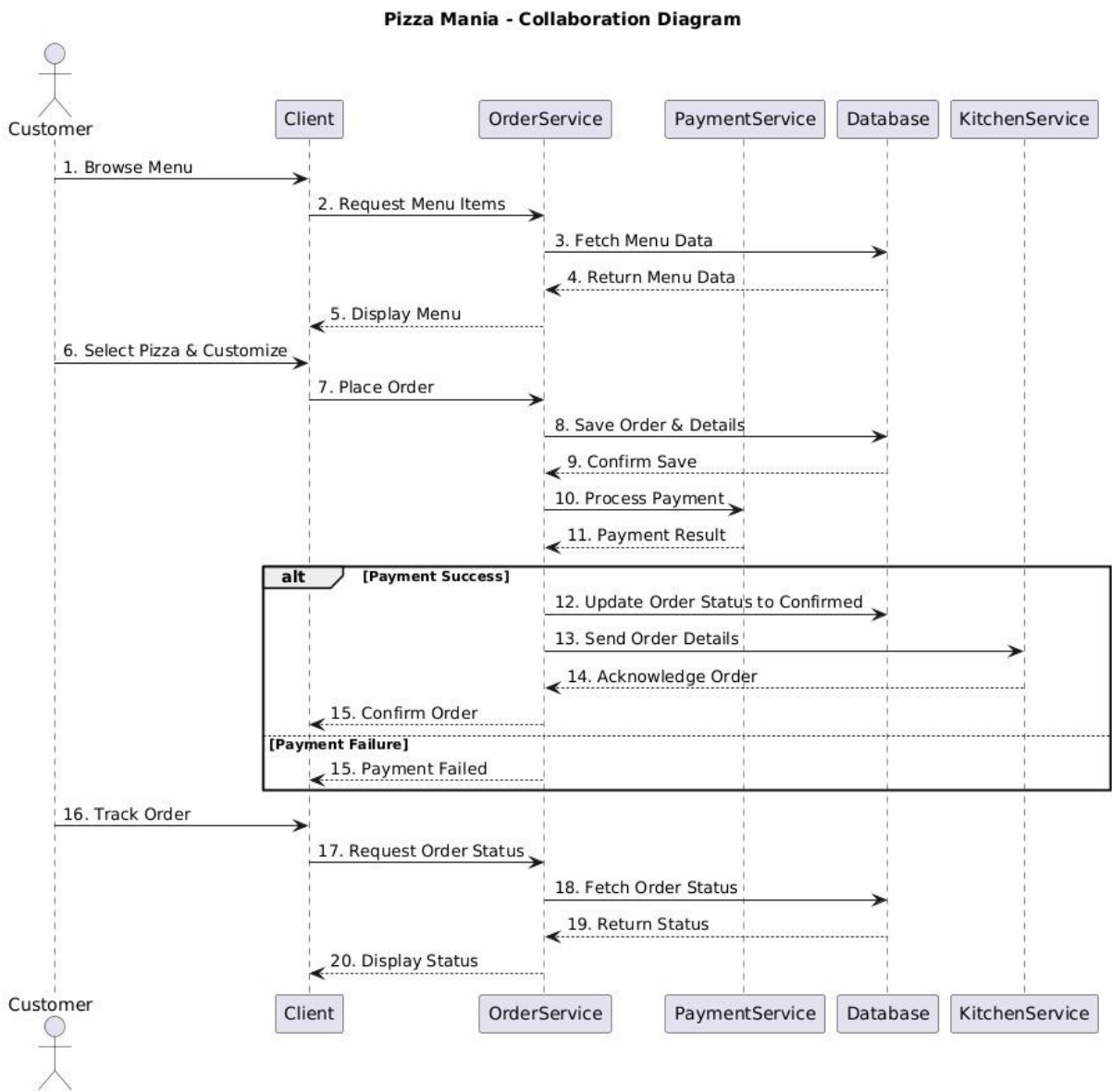
State Chart Diagram:



Sequence Diagram:



Collaboration Diagram:



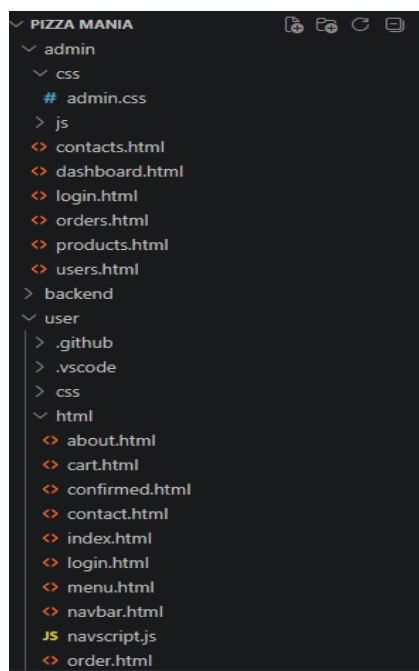
Implementation

1. Modules Overview

- **Customer Side:**
 - **Home/About:** Welcome page and brand introduction.
 - **Menu:** Real-time pizza listing fetched from MongoDB.
 - **Cart & Checkout:** Order selection, quantity management, and final submission.
 - **Contact:** Inquiry form for customer feedback and support.
- **Admin Side:**
 - **Dashboard:** Summary of sales, order volume, and user growth.
 - **Management Tools:** Interfaces to **CRUD** (Create, Read, Update, Delete) products, track orders, and view registered user data.

2. Redirect & Authentication Logic

- **Security:** Uses **Token-Based Authentication**. Upon login, an adminToken is stored in the browser's **LocalStorage**.
- **Access Control:** All admin pages run a script to verify the token. If the user is not an authorized admin, they are automatically **redirected** to the login page.
- **Navigation Logic:**
 - * **Login:** Redirects to the Dashboard.
 - **Checkout:** Redirects to a Success page after order placement.
 - **Logout:** Clears the adminToken and redirects to the Login screen immediately.



MongoDB Connection

```
6 // -----
7 const mongoose = require("mongoose");
8
9 const connectDB = async () => {
10   try {
11     const conn = await mongoose.connect(process.env.MONGO_URI);
12     console.log(`✅ MongoDB Connected`);
13   } catch (error) {
14     console.error(`❌ MongoDB Connection Error: ${error.message}`);
15     process.exit(1); // Stop the server if DB connection fails
16   }
17 };
18
19 module.exports = connectDB;
20
```

Cart schema model

```
const mongoose = require("mongoose");
✓ const cartItemSchema = new mongoose.Schema({
✓   product: {
     type: mongoose.Schema.Types.ObjectId,
     ref: "Product",
     required: true,
   },
   name: { type: String, required: true }, // snapshot at time of adding
   price: { type: Number, required: true }, // snapshot at time of adding
   image: { type: String },
   qty: { type: Number, default: 1, min: 1 },
});
✓ const cartSchema = new mongoose.Schema(
  {
    ✓ user: {
        type: mongoose.Schema.Types.ObjectId,
        ref: "User",
        required: true,
        unique: true, // one cart document per user
      },
    items: [cartItemSchema],
  },
  { timestamps: true }
);

// — Virtual: compute cart totals without storing them —————
✓ cartSchema.virtual("subtotal").get(function () {
  | return this.items.reduce((sum, item) => sum + item.price * item.qty, 0);
});

module.exports = mongoose.model("Cart", cartSchema);
```

Get Cart api Controller

```
// — GET /api/cart —————
// Returns the current user's cart
const getCart = async (req, res) => {
  try {
    let cart = await Cart.findOne({ user: req.user._id });

    if (!cart) {
      // No cart yet – return empty cart
      return res.json({ items: [], subtotal: 0 });
    }

    const subtotal = cart.items.reduce(
      (sum, item) => sum + item.price * item.qty, 0
    );

    res.json({ items: cart.items, subtotal });
  } catch (error) {
    res.status(500).json({ message: "Server error: " + error.message });
  }
};
```

Add to Cart api Controller

```
// — POST /api/cart —————
// Adds an item to the cart (or increases qty if already present)
const addToCart = async (req, res) => {
  try {
    const { productId, qty = 1 } = req.body;

    if (!productId) {
      return res.status(400).json({ message: "Product ID is required" });
    }

    // Verify the product exists
    const product = await Product.findById(productId);
    if (!product) {
      return res.status(404).json({ message: "Product not found" });
    }
    if (!product.isAvailable) {
      return res.status(400).json({ message: "Product is not available" });
    }

    // Find or create the cart for this user
    let cart = await Cart.findOne({ user: req.user._id });
    if (!cart) {
      cart = new Cart({ user: req.user._id, items: [] });
    }

    // Check if item already in cart
    const existing = cart.items.find(
      (item) => item.product.toString() === productId
    );

    if (existing) {
      existing.qty += qty; // increase quantity
    } else {
      cart.items.push({
        product: product._id,
        name: product.name,
        price: product.price,
        image: product.image,
        qty,
      });
    }

    await cart.save();

    const subtotal = cart.items.reduce(
      (sum, item) => sum + item.price * item.qty, 0
    );

    res.json({ message: "Item added to cart!", items: cart.items, subtotal });
  } catch (error) {
    res.status(500).json({ message: "Server error: " + error.message });
  }
};
```

Cart Routes

```
// All cart routes are protected – user must be logged in
router.use(protect);

router.get("/", getCart);
router.post("/", addToCart);
```

Server File

```
// Enable CORS – allows the frontend (running on a different port) to call this API
app.use(cors({
  origin: "*", // Allow all origins in development
  methods: ["GET", "POST", "PUT", "DELETE"],
  allowedHeaders: ["Content-Type", "Authorization"],
}));

// Parse incoming JSON request bodies
app.use(express.json());

// — Routes —
app.use("/api/auth", require("./routes/authRoutes"));
app.use("/api/products", require("./routes/productRoutes"));
app.use("/api/cart", require("./routes/cartRoutes"));
app.use("/api/orders", require("./routes/orderRoutes"));
app.use("/api/contact", require("./routes/contactRoutes"));

// — Health Check —
app.get("/", (req, res) => {
  res.json({
    message: "Pizza Mania API is running!",
    version: "1.0.0",
    endpoints: {
      auth: "/api/auth",
      products: "/api/products",
      cart: "/api/cart",
      orders: "/api/orders",
    },
  });
});

// — 404 Handler —
app.use((req, res) => {
  res.status(404).json({ message: `Route not found: ${req.method} ${req.url}` });
});

// — Start Server —
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`\n🍕 Pizza Mania API running at http://localhost:${PORT}`);
  console.log(`Environment: ${process.env.NODE_ENV}`);
  console.log(`Press Ctrl+C to stop\n`);
});
```

Home page

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>Pizza Mania</title>
    <link rel="stylesheet" href="../../css/home.css" />
  </head>
  <body>
    <!-- Navbar load here -->
    <div id="navbar"></div>

    <!-- Hero Section -->
    <section class="hero">
      <div class="hero-text">
        <h1>Welcome To <br />Pizza Mania</h1>
        <p>
          Where every slice is loaded with flavor,<br>
          every bite is pure happiness,<br>
          and your cravings finally meet their match.
        </p>
        <a href="menu.html" class="btn">Explore Now</a>
      </div>

      <div class="hero-image">
        
      </div>
    </section>

    <!-- config.js provides API_URL, isLoggedIn(), updateNavbar() -->
    <script src="../../javascript/config.js"></script>
    <script>
      fetch("navbar.html")
        .then((response) => response.text())
        .then((data) => {
          document.getElementById("navbar").innerHTML = data;
          updateNavbar();
        });
    </script>
  </body>
</html>
```

Result and Analysis

Login page

 **Pizza Mania**

Home About Us Contact Menu Cart Login

Welcome Back 🍕

Login to your Pizza Mania account

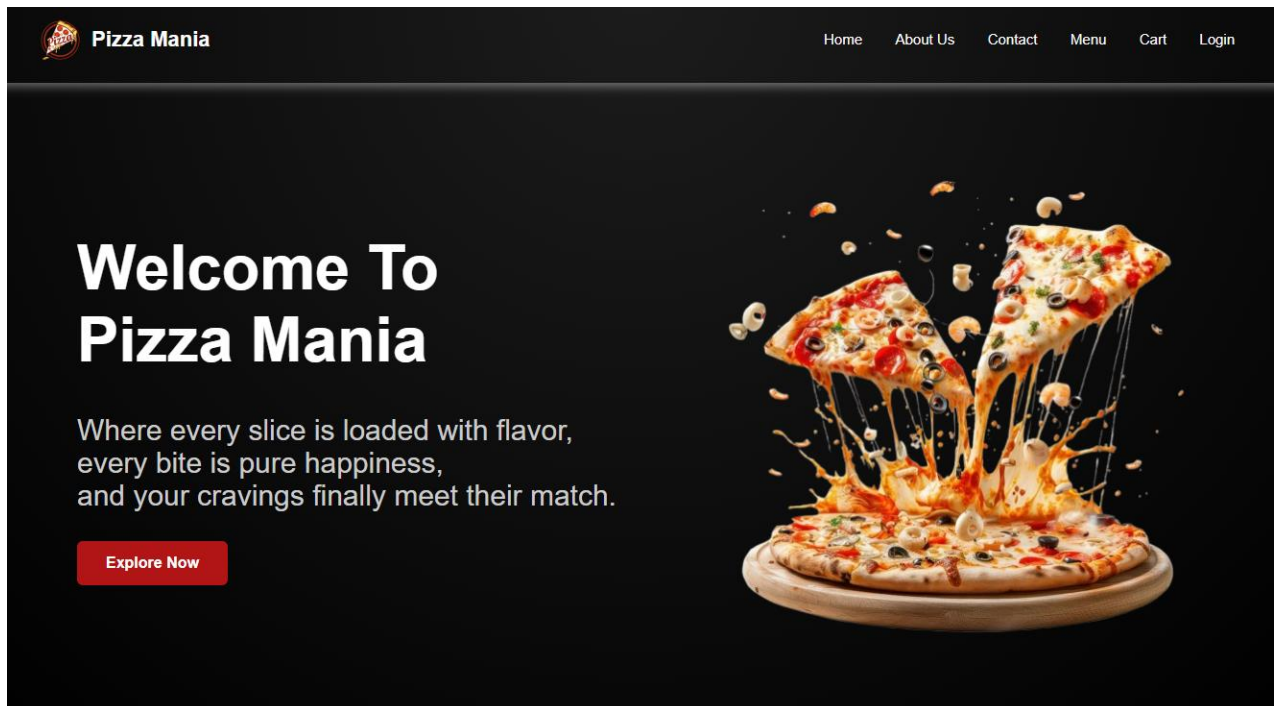
Email Address

Password

Login

Don't have an account? [Register here](#)

Home page



Menu page

 **Pizza Mania**

HomeAbout UsContactMenuCart

Our Special Menu

Handcrafted pizzas made with fresh ingredients & real passion



BBQ Chicken

Grilled chicken tossed in smoky BBQ sauce, onions, and cheese baked to perfection.

₹349

[Add to Cart](#)



Pepperoni

loaded with spicy pepperoni slices and melted mozzarella on a crispy, golden crust

₹229

[Add to Cart](#)



Vegetarian

A colorful mix of fresh vegetables, olives, capsicum, tomatoes, and premium cheese

₹199

[Add to Cart](#)

Cart page

 **Pizza Mania**

HomeAbout UsContactMenuCart

Your Cart

Product	Price	Subtotal	Remove
 BBQ Chicken ₹349 each	- 1 +	₹349	Remove
 Mediterranean Veg Pizza ₹279 each	- 2 +	₹558	Remove

Subtotal:

₹907

Tax (9%):

₹82

Total:

₹989

[Proceed to Checkout](#)

Check out page

 **Pizza Mania**

[Home](#) [About Us](#) [Contact](#) [Menu](#) [Cart](#) 

Checkout

Billing Details

Mohammad Rashid Salim

mrrashid9130@gmail.com

9130818332

camp, pune

pune

444001

PLACE ORDER

Your Order

BBQ Chicken × 1	₹349
Mediterranean Veg Pizza × 2	₹558
Subtotal:	₹907
Tax (9%):	₹82
Total:	₹989

Order Success Page

 **Pizza Mania**

[Home](#) [About Us](#) [Contact](#) [Menu](#) [Cart](#) 



Order Confirmed!

Thank you for your order. Your delicious pizzas are on the way 🍕

Order Summary

BBQ Chicken × 1	₹349
Mediterranean Veg Pizza × 1	₹279
Subtotal	₹628
Tax	₹57
Total	₹685

Delivery Details

NAME

Mohammad Rashid Salim

ADDRESS

camp, pune

CITY

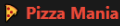
pune

PHONE

9130818332

Continue Shopping

Admin Dashboard Page


Admin Panel

Dashboard

Products

Orders

Users

Enquiries

Logout

Dashboard

Welcome back, Admin

TOTAL ORDERS

10

TOTAL REVENUE

₹4,788

TOTAL USERS

6

PRODUCTS

6

Recent Orders

View All

ORDER ID	CUSTOMER	ITEMS	TOTAL	STATUS	DATE
#F68E04	Affan sheikh	1 item	₹271	CONFIRMED	31/3/2026
#EE2AB8	Mohammad Rashid Salim	1 item	₹326	CANCELLED	30/3/2026
#9718E2	Mohammad Rashid Salim	1 item	₹326	CONFIRMED	30/3/2026
#97181F	Mohammad Rashid Salim	1 item	₹326	PREPARING	30/3/2026
#971797	Mohammad Rashid Salim	2 items	₹934	CONFIRMED	30/3/2026

Products Management page


Admin Panel

Dashboard

Products

Orders

Users

Enquiries

Logout

Products

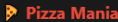
Manage the pizza menu — add, edit or delete items

All Products

+ Add Product

IMAGE	NAME	CATEGORY	PRICE	POPULAR	AVAILABLE	ACTIONS
	BBQ Chicken	NON-VEG	₹349	★ Yes	YES	Edit Delete
	Pepperoni	VEG	₹229	—	YES	Edit Delete
	Vegetarian	VEG	₹199	★ Yes	YES	Edit Delete
	Truffle Mushroom Pizza	VEG	₹299	—	YES	Edit Delete
	Mediterranean Veg Pizza	VEG	₹279	—	YES	Edit Delete
	Meat Lovers	NON-VEG	₹249	★ Yes	YES	Edit Delete

Order Management Module

 Pizza Mania

Admin Panel

 Dashboard

 Products

 Orders

 Users

 Enquiries

 Logout

Orders

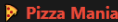
View and manage all customer orders

All Orders

Search by name or ID...

ORDER ID	CUSTOMER	ITEMS	TOTAL	STATUS	DATE	UPDATE STATUS
#0FF0A9	Mohammad Rashid Salim	BBQ Chicken x1, Mediterranean Veg Pizza x2	₹989	CONFIRMED	1/4/2026	Confirmed
#0FF05B	Mohammad Rashid Salim	BBQ Chicken x1, Mediterranean Veg Pizza x1	₹685	DELIVERED	1/4/2026	Delivered
#F68E04	affan shaikh	Meat Lovers x1	₹271	CONFIRMED	31/3/2026	Confirmed
#EE2AB8	Mohammad Rashid Salim	Truffle Mushroom Pizza x1	₹326	CANCELLED	30/3/2026	Cancelled
#9718E2	Mohammad Rashid Salim	Truffle Mushroom Pizza x1	₹326	CONFIRMED	30/3/2026	Confirmed
#97181F	Mohammad Rashid Salim	Vegetarian x1	₹326	PREPARING	30/3/2026	Preparing
#971797	Mohammad Rashid Salim	Vegetarian x1, Mediterranean Veg Pizza x2	₹934	CONFIRMED	30/3/2026	Confirmed
#96CEEA	Mohammad Rashid Salim	Truffle Mushroom Pizza x2, Vegetarian x1	₹978	DELIVERED	30/3/2026	Delivered
#96CE47	Mohammad Rashid Salim	Margherita x1	₹271	CANCELLED	30/3/2026	Cancelled
#65E062	Md Aatif Shaikh	Margherita x1	₹271	DELIVERED	29/3/2026	Delivered
#31F84F	Md Aatif Shaikh	Margherita x1	₹271	DELIVERED	29/3/2026	Delivered

User Management Page

 Pizza Mania

Admin Panel

 Dashboard

 Products

 Orders

 Users

 Enquiries

 Logout

Users

All registered customer accounts

All Users

Search by name or email...

#	NAME	EMAIL	ROLE	JOINED
1	Mohammad Rashid	mrrashid@gmail.com	USER	31 Mar 2026
2	Affan sheikh	affusk09@gamil.com	USER	31 Mar 2026
3	Mohammad Rashid Salim	mrrashid9130@gmail.com	USER	30 Mar 2026
4	Test	aatifmd7218@gmail.com	USER	29 Mar 2026
5	Test	test@gmail.com	USER	29 Mar 2026
6	Admin	admin@pizzamania.com	ADMIN	1 Jan 2024

Conclusion

The **Pizza Mania** project concludes as a successful implementation of a modern, web-based food ordering system that effectively replaces traditional manual processes with a streamlined digital solution. Built using the **Node.js, Express, and MongoDB** stack, the platform provides an intuitive interface for customers to browse menus and place orders while offering business owners a powerful **Admin Panel** for real-time management of inventory and sales. By centralizing data and automating the order-to-delivery pipeline, the project reduces human error, improves operational efficiency, and significantly enhances the overall customer experience. Ultimately, this system serves as a scalable foundation for a digital restaurant business, demonstrating how technology can be used to meet the demands of the modern food industry.

Future Scope

- **Payment Gateway Integration:** Adding secure online payment options such as **Razorpay or Stripe** to allow cashless transactions.
- **Real-time Order Tracking:** Implementing **Google Maps API** to provide customers with live updates on their pizza delivery status.
- **Mobile Application Development:** Converting the web platform into native **Android and iOS** applications for better accessibility.
- **Automated Notifications:** Integrating **SMS and Email alerts** to notify customers automatically when their order is confirmed or out for delivery.
- **AI-Driven Recommendations:** Using machine learning to analyze user order history and suggest their favorite pizzas or personalized deals.

8. References

1. **Node.js Documentation** – <https://nodejs.org/docs> (Reference for the server-side runtime environment)
2. **Express.js Documentation** – <https://expressjs.com> (Reference for web application framework and routing)
3. **MongoDB Documentation** – <https://www.mongodb.com/docs> (Reference for NoSQL database and data modeling)
4. **W3Schools (HTML, CSS, JS)** – <https://www.w3schools.com> (Reference for frontend design and UI styling)
5. **MDN Web Docs** – <https://developer.mozilla.org> (Reference for modern JavaScript practices and Web APIs)
6. **YouTube Tutorials** (Reference for MERN stack project structure and implementation)